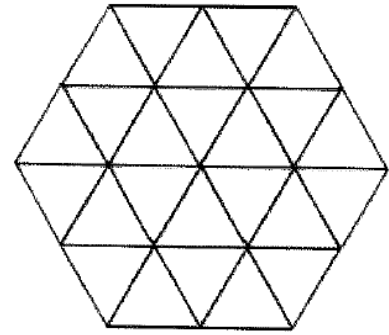


In-class Task (Csaba Gabor): You are given an inner hexagon (made up of 6 small triangles), and around each of the 6 outer vertices of the hexagon you construct another hexagon to wind up with a total of 7 complete hexagons utilizing 24 of the small triangles in total. In other words, the outer perimeter has 18 triangles. In addition, for each of the three main directions for this amalgamation of triangles, there are 4 rows of either 5 or 7 triangles. That is, there are 12 distinct rows of triangles.



Find a way to label each small triangle with an integer from 1-6 where none of the seven hexagons has two triangles with the same number.

Dr. G's considerations:

Your program is not expected to draw triangles or hexagons. However, it must have some way of identifying each of the triangles. So, you will have to devise some indexing scheme to decide which is the zero triangle, which is triangle with index 1, and so forth. As long as each triangle has a unique index, it would be a valid indexing.

What is important is what sets of triangles constitute a hexagon, or a row of triangles. If you make a dictionary of each such set, then your code should run just fine. Besides representing your puzzle as a string or list, it's the only data structure you'll need.

Use a period(.) to represent an unfilled position in the puzzle. This is because _ and spaces tend to merge when displayed while the periods will stay distinct.

Draw an image of the triangle amalgamation and fill it in with the solution.

Define this CSP and list what you need to make and pass to your backtracking function:

Variables: 24 triangles → a string with length 24. Ex) "....."

Domain: $D_i = \{1, 2, 3, 4, 5, 6\}$

Data structures:

assignment = a string

variables = index of the string. Each char will be . or a one-digit integer on [1, 6]

csp: each hexagon has 6 distinct integers on [1, 6] → make a lookup table.

Ex) csp_table = [[0, 1, 2, 6, 7, 8], [2, 3, 4, 8, 9, 10], ...] # indices of assignment

Lab Specification:

File Name: LastName_FirstInitial_L2.py

Sample input:

- "....." (24 dots)
- "1.....5.....3....." (some inputs) means

1	.	.	.	.	.
.	.	5	.	.	.
.	.	3	.	.	.
.	.	.	.	.	.

Sample outputs:

```

▶▶ 24-char(. and 1-6) input: .....
    1 2 3 1 2
    1 4 5 6 4 5 1
    2 6 3 1 2 3 6
    2 4 5 4 6

```

```
123121456451263123624546
```

```
True
```

```

▶▶ 24-char(. and 1-6) input: 6.....34...1.....2..4...
    6 1 2 1 3
    1 3 4 5 6 4 1
    5 6 2 1 3 2 5
    3 4 5 4 6

```

```
612131345641562132534546
```

```
True
```

HW) Sudoku lab setting will be similar to the today's task, such as using a string for assignment. Think about how you will design your program to solve a 9 by 9 sudoku.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Sample input:

```
"..9.8.26.314..69.7...3..4.5..592....9..6..5....436..9.6..7...8.76..192.31.9.7.."
```